

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский ядерный университет «МИФИ»

Технологический институт –

филиал федерального государственного автономного образовательного учреждения высшего
образования «Национальный исследовательский ядерный университет «МИФИ»
(ТИ НИЯУ МИФИ)

**КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И ПРИКЛАДНОЙ МАТЕМАТИКИ**

ОТЧЕТ
по производственной практике
по направлению бакалавриата
09.03.01 «Информатика и вычислительная техника»

**Руководитель
от предприятия**

(должность)

(подпись)

И.Д. Смирнов

(И.О.Фамилия)

**Руководитель
от вуза**

И.о. зав.кафедрой

(должность)

(подпись)

Н.В. Чупракова

(И.О.Фамилия)

**Студент
ИВТ-31Д**

(группа)

(подпись)

И.А. Себульке

(И.О.Фамилия)

Отчет защищен с оценкой « _____ » « _____ » _____ 2024 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ГЛАВА 1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ, ВЫБОР ИНСТРУМЕНТОВ	4
1.1 Функции программного обеспечения.....	4
1.2 Выбор инструментов.....	4
ГЛАВА 2 СОЗДАНИЕ ПРИЛОЖЕНИЯ	6
2.1 Разработка SQL процедуры для обхода таблиц	6
2.2 Визуальное проектирование приложения, проверка работоспособности	7
ЗАКЛЮЧЕНИЕ	9
СПИСОК ЛИТЕРАТУРЫ.....	10
ПРИЛОЖЕНИЕ А	11
ПРИЛОЖЕНИЕ Б.....	13

ВВЕДЕНИЕ

ФГУП «Комбинат «Электрохимприбор» – одно из крупнейших российских машиностроительных предприятий, производящих сложные высокотехнологичные изделия специального назначения. На предприятии существует большое количество взаимосвязанных производственных процессов, так же называемых бизнес-процессами. Описание и оптимизация бизнес-процессов является важнейшим условием эффективного функционирования предприятия. Для их проектирования, разработки и дальнейшего внедрения могут вводиться комплексы определенных мероприятий: подбор необходимого персонала, повышение квалификации или переподготовка имеющихся кадров, реорганизация отделов и, в том числе, создание программного обеспечения, ориентированного специально под решение производственных задач предприятия. Бизнес-процессы не только могут быть различными, но и изменяются во времени ввиду изменения условий их функционирования, либо в процессе их эксплуатации выявляются объективные проблемы, требующие оперативного решения.

В качестве задания при прохождении практики на предприятии ФГУП «Комбинат «Электрохимприбор» была поставлена задача разработки специального веб-приложения – глоссария и его серверной части для общения с базой данных, которая предполагала просмотр и сбор информации о различных названиях колонок таблиц и их расшифровок, образовавшихся при переходе с СУБД MSSQL на PostgreSQL из-за того, что ранее колонки таблиц баз данных имели русскоязычные названия, а сейчас – английские.

Для достижения поставленной цели необходимо выполнить следующие задачи:

1. Проанализировать предметную область;
2. Выделить требования к разрабатываемому ПО;
3. Создать инструмент обхода базы данных;
4. Спроектировать дизайн веб-приложения;
5. Запрограммировать веб-приложение;
6. Протестировать веб-приложение.

ГЛАВА 1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ, ВЫБОР ИНСТРУМЕНТОВ

1.1 Функции программного обеспечения

На основе технического задания можно обозначить основные требования к разрабатываемой системе. Известно, что в базе внутри схем будут храниться структуры различных таблиц вместе с названиями колонок, а также их описанием. Как результат, программное обеспечение должно обойти все обозначенные пользователем схемы и их таблицы, собрав информацию о связях полей, их описаниях и прочих технических данных, а затем, на основе полученного описания, попытаться угадать/выделить русскоязычное название, которое было использовано аналогичным полем в MSSQL.

Разрабатываемое веб-приложение должно выполнять следующие функции:

1. Просмотр информации о собранных колонках баз данных;
2. Просмотр и возможность копирования возможных русских переводов;
3. Поиск и фильтрация записей.

Также на стороне PostgreSQL должна быть реализована процедура, выполняющая обход всех схем, названия которых начинаются с выбранной пользователем строки, а также содержащихся в них таблиц с полями.

1.2 Выбор инструментов

Инструментарий представляет собой PostgreSQL и Python с Django. PostgreSQL представляет собой мощную, открытую систему управления объектно-реляционными базами данных. Основным языком запросов является SQL, с расширениями для поддержания процедур и функций. Postgres используется для работы с базами данных различных масштабов — от небольших до крупных корпоративных приложений. Для управления базами данных и выполнения административных задач использовался pgAdmin 4, предоставляющий разнообразные инструменты для разработчиков и администраторов баз данных.

Для написания серверной части приложения использовался язык программирования Python версии 3.7.3, являющийся мощным, высокоуровневым языком программирования, который благодаря своей читаемости и лаконичности широко используется в веб-разработке, науке о данных, автоматизации и других областях.

В качестве веб-фреймворка для создания серверной части приложения был выбран Django версии 4.2.5, высокоуровневый веб-фреймворк, написанный на Python, который

способствует быстрой разработке и чистому, прагматичному дизайну, а также библиотека для него – Django Rest Framework. Django включает множество функций, среди которых есть и ORM (Object-Relational Mapping), что позволяет значительно ускорить процесс разработки и сосредоточиться на логике приложения, а не на написании повторяющегося кода.

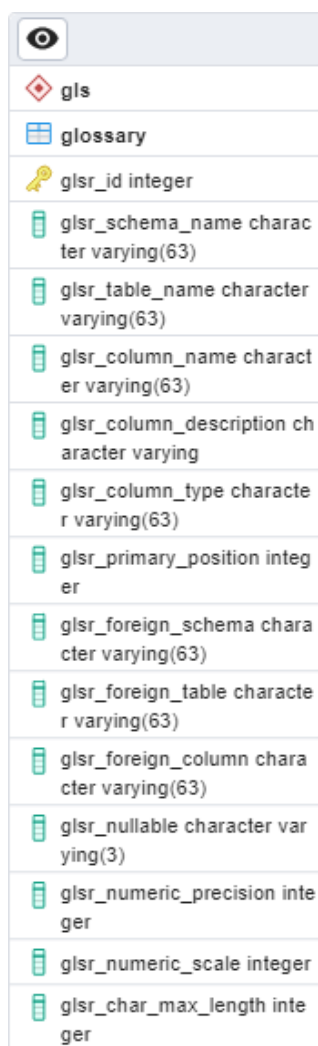
ГЛАВА 2 СОЗДАНИЕ ПРИЛОЖЕНИЯ

Выполнение задания включает в себя следующие этапы:

1. Разработка SQL процедуры для обхода таблиц;
2. Визуальное проектирование приложения;
3. Программирование событий, обеспечивающих работоспособность приложения.

2.1 Разработка SQL процедуры для обхода таблиц

На основе анализа технического задания была разработана SQL процедура, в которую в качестве параметра можно передать строку, с которой должны начинаться схемы, которые необходимо проанализировать. Результатом выполнения процедуры является заполненная таблица, структура которой представлена на Рисунке 2.1.



The image shows a screenshot of a database table structure for a table named 'glossary'. The table has 15 columns. The first column is 'glsr_id' of type 'integer' and is marked as the primary key with a yellow key icon. The remaining 14 columns are of type 'character varying' with various lengths or are 'integer' type. Each column has a green icon to its left. The table is displayed in a light blue header and white body format.

gls
glossary
glsr_id integer
glsr_schema_name character varying(63)
glsr_table_name character varying(63)
glsr_column_name character varying(63)
glsr_column_description character varying
glsr_column_type character varying(63)
glsr_primary_position integer
glsr_foreign_schema character varying(63)
glsr_foreign_table character varying(63)
glsr_foreign_column character varying(63)
glsr_nullable character varying(3)
glsr_numeric_precision integer
glsr_numeric_scale integer
glsr_char_max_length integer

Рисунок 2.1 – Таблица glossary с результатами выполнения процедуры

2.2 Визуальное проектирование приложения, проверка работоспособности

В ходе работы был разработано одностраничное веб-приложение для просмотра сохраненных данных, интерфейс которого представлен на Рисунке 2.2.

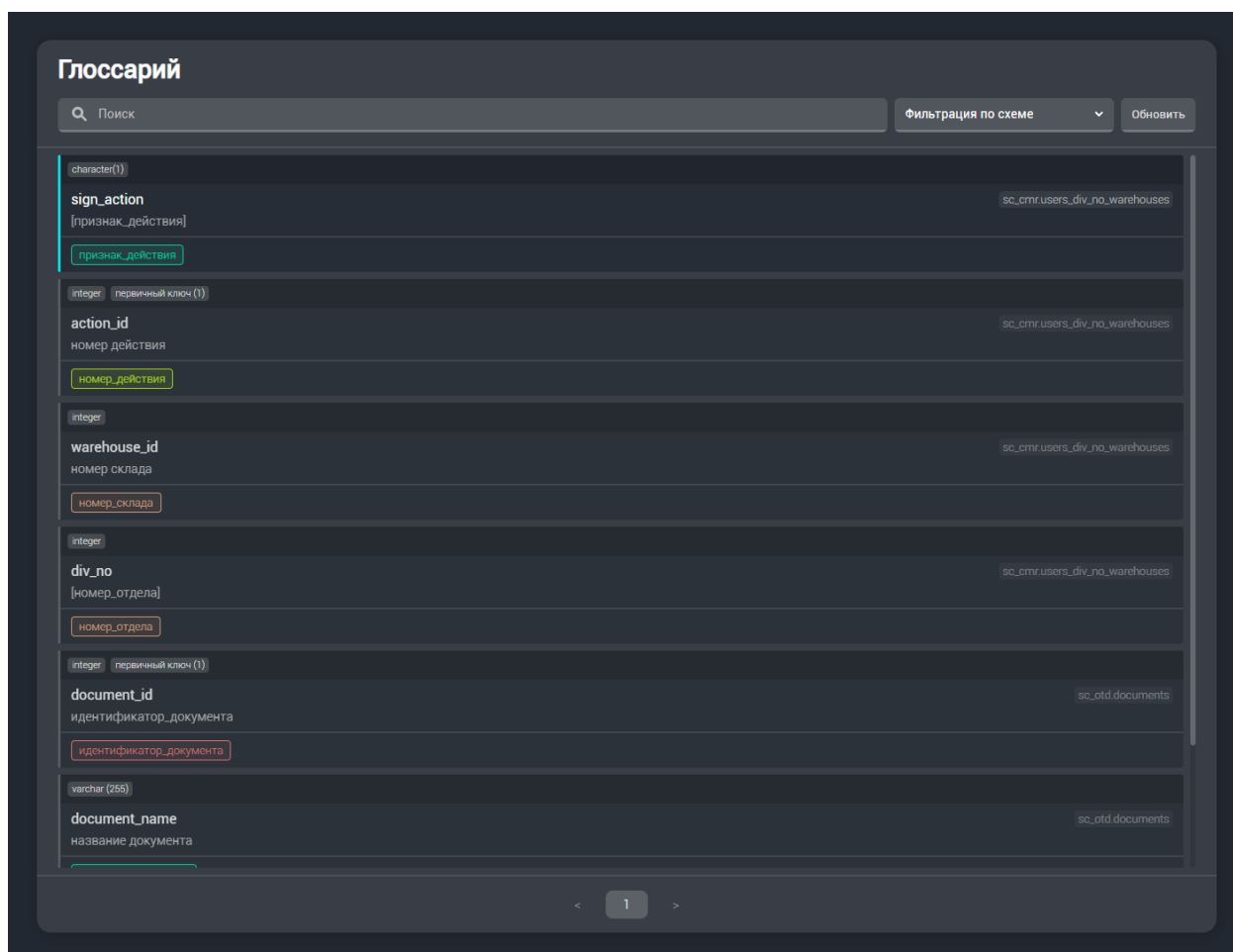


Рисунок 2.2 – Интерфейс главного экрана приложения

Из предложенного функционала имеется возможность выполнения поиска по любым атрибутам записей, а также выполнение фильтрации по схемам, что позволяет исключить записи из ненужных схем и отобразить только необходимые.

Цветные ярлыки, присущие записям, являются предложенными вариантами их перевода на русский язык, которые вычисляются из описания колонки по следующему принципу: слова, заключенные в квадратные скобки, абсолютно точно принимаются в качестве перевода, т.к. именно такую пометку чаще всего использовали при переносе базы данных на PostgreSQL. Также двух- и трёхсловные описания считаются за возможный перевод. При нажатии на такой ярлык происходит копирование текста (Рисунок 2.3).

Приложение поддерживает пагинацию (переключение между страницами), что позволяет снизить нагрузку на сервер за счет минимизирования отправляемых данных. Поиск, фильтрация и переключение страниц, при этом, не вызывает обновление страницы.

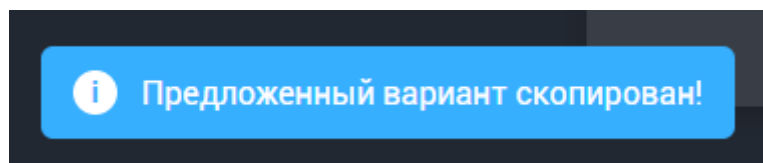


Рисунок 2.3 – Сообщение об успешном копировании предложенного перевода

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики предприятие ФГУП «Комбинат «Электрохимприбор» в качестве задания поставило разработку веб-приложения – глоссария и его серверной части для общения с базой данных, которая предполагала просмотр и сбор информации о различных названиях колонок таблиц и их расшифровок, образовавшихся при перехода с СУБД MSSQL на PostgreSQL.

Как результат было разработано веб-приложение, удовлетворяющее всем условиям задания, которое прошло проверку работоспособности. Веб-приложение осуществляет корректную работу с данными, правильно отображая и фильтруя информацию. Поставленные на прохождение практики задачи были успешно решены.

СПИСОК ЛИТЕРАТУРЫ

1. About Postgresql [Электронный ресурс]. - Режим доступа: <https://www.postgresql.org/about/> (дата обращения: 04.07.2024).
2. Документация PostgreSQL (The Information Schema) [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/current/information-schema.html> (дата обращения: 02.07.2024).
3. Разработка на Django [Электронный ресурс]. - Режим доступа: <https://www.djangoproject.com/foundation/> (дата обращения: 04.07.2024).

ПРИЛОЖЕНИЕ А

Код SQL процедуры

```
CREATE OR REPLACE PROCEDURE gls_new.generate_for_schemas("aSchemaName" character varying
DEFAULT 'sc_%')
LANGUAGE plpgsql
```

```
AS $$
begin
```

```
DROP TABLE IF EXISTS temp_primary_keys;
DROP TABLE IF EXISTS temp_foreign_keys;
```

```
-- Create temp tables
```

```
CREATE TEMPORARY TABLE temp_primary_keys(
    id integer NOT NULL GENERATED ALWAYS AS IDENTITY,
    table_schema varchar(63),
    table_name varchar(63),
    column_name varchar(63),
    "position" integer
);
```

```
CREATE TEMP TABLE temp_foreign_keys(
    id integer NOT NULL GENERATED ALWAYS AS IDENTITY,
    table_schema varchar(63),
    table_name varchar(63),
    column_name varchar(63),
    foreign_table_schema varchar(63),
    foreign_table_name varchar(63),
    foreign_column_name varchar(63)
);
```

```
-- Get info about all primary keys
```

```
INSERT INTO temp_primary_keys (table_schema, table_name, column_name, position) (
    SELECT DISTINCT
        kcu.table_schema,
        kcu.table_name,
        kcu.column_name AS key_column,
        kcu.ordinal_position AS position
    FROM information_schema.table_constraints tco
    JOIN information_schema.key_column_usage kcu
        ON kcu.constraint_name = tco.constraint_name
        AND kcu.constraint_schema = tco.constraint_schema
    WHERE
        tco.constraint_type = 'PRIMARY KEY'
        AND tco.constraint_schema LIKE "aSchemaName"
);
```

```
-- Get info about all foreign keys
```

```
INSERT INTO temp_foreign_keys (table_schema, table_name, column_name, foreign_table_schema,
foreign_table_name, foreign_column_name) (
    SELECT DISTINCT
        tc.table_schema,
        tc.table_name,
        kcu.column_name,
        ccu.table_schema AS foreign_table_schema,
        ccu.table_name AS foreign_table_name,
        ccu.column_name AS foreign_column_name
    FROM information_schema.table_constraints tc
    JOIN information_schema.key_column_usage kcu
        ON tc.constraint_name = kcu.constraint_name
        AND tc.table_schema = kcu.table_schema
);
```

```

        JOIN information_schema.constraint_column_usage ccu
            ON ccu.constraint_name = tc.constraint_name
    WHERE
        tc.constraint_type = 'FOREIGN KEY' AND
        ccu.table_schema LIKE "aSchemaName"
);

-- Create glossary
TRUNCATE TABLE gls_new.glossary RESTART IDENTITY;

INSERT INTO gls_new.glossary (glsr_column_name, glsr_column_description, glsr_column_type,
glsr_schema_name, glsr_table_name, glsr_primary_position, glsr_foreign_schema, glsr_foreign_table,
glsr_foreign_column, glsr_nullable, glsr_numeric_precision, glsr_numeric_scale, glsr_char_max_length) (
    SELECT
        c.column_name,
        regexp_replace(pgd.description, '(\r|[\x0a]\s*)', ' ', 'g'),
        c.data_type,
        c.table_schema,
        c.table_name,
        pkey.position,
        fkey.foreign_table_schema,
        fkey.foreign_table_name,
        fkey.foreign_column_name,
        c.is_nullable,
        c.numeric_precision,
        c.numeric_scale,
        c.character_maximum_length
    FROM (pg_catalog.pg_statio_all_tables st
        INNER JOIN pg_catalog.pg_description pgd
            ON pgd.objoid = st.relid
        INNER JOIN information_schema.columns c
            ON pgd.objsubid = c.ordinal_position
            AND c.table_schema = st.schemaname
            AND c.table_name = st.relname
    )
    LEFT JOIN temp_foreign_keys fkey
        ON fkey.table_schema = c.table_schema
        AND fkey.table_name = c.table_name
        AND fkey.column_name = c.column_name
    LEFT JOIN temp_primary_keys pkey
        ON pkey.table_schema = c.table_schema
        AND pkey.table_name = c.table_name
        AND pkey.column_name = c.column_name
    WHERE
        c.table_schema LIKE "aSchemaName"
);

DROP TABLE IF EXISTS temp_primary_keys;
DROP TABLE IF EXISTS temp_foreign_keys;

end; $$;

```

ПРИЛОЖЕНИЕ Б

Код веб-приложения

```
(function () {
  function HashString(string) {
    let result = string.length * 1475292;

    for (let i = 0; i < string.length; i++) {
      let charCode = string.charCodeAt(i);
      result = ((result << 5) - result) + charCode;
      result |= 0;
    }

    return result;
  }

  class GaussFilter {
    constructor() {
      this.numBuckets = 128;
      this.items = new Int8Array(this.numBuckets);

      for (let i = 0; i < this.numBuckets; i++) {
        this.items[i] = 0;
      }
    }

    add(value) {
      const bucket = Math.abs(HashString(value.toString().toLowerCase())) % this.numBuckets;
      this.items[bucket] = 1;
    }

    exists(value) {
      const bucket = Math.abs(HashString(value.toString().toLowerCase())) % this.numBuckets;
      return this.items[bucket] == 1;
    }
  }

  class QueryBuilder {
    constructor(url) {
      this.queryUrl = url ?? "";
      this.hasAnyParams = false;
    }

    addParam(name, value) {
      const trimName = encodeURIComponent(name.toString().trim());
      const trimValue = encodeURIComponent(value.toString().trim());

      if (trimValue.length > 0) {
        if (!this.hasAnyParams) this.queryUrl += "?";
        else this.queryUrl += "&";

        this.queryUrl += `${trimName}=${trimValue}`;
        this.hasAnyParams = true;
      }

      return this;
    }

    getUrl() {
      return this.queryUrl;
    }
  }
})
```

```

class RequestSettings {
  constructor() {
    this.Reset();
  }

  Reset() {
    this.search = this.ParseQueryParam("search", "");
    this.schema = this.ParseQueryParam("schema", "");
    this.page = Math.max(this.ParseQueryParam("page", 1), 0);
  }

  ParseQueryParam(paramName, defaultValue) {
    let search = window.location.search.substring(1);
    let parameters = search.split('&');

    for (let i = 0; i < parameters.length; i++) {
      let parameter = parameters[i];
      let splitParameter = parameter.split('=');

      if (splitParameter[0] == paramName) {
        if (splitParameter.length == 1) {
          // Assume this is a boolean type value
          return true;
        } else {
          // Return the value set after the '=' token
          let decodedString = decodeURIComponent(splitParameter[1]);

          if (typeof defaultValue === 'number') {
            // Convert string to a number value
            return +decodedString;
          }

          return decodedString;
        }
      }
    }

    // Couldn't find parameter, return default value
    return defaultValue;
  }

  UpdateHistoryURL() {
    if (history.pushState) {
      const queryBuilder = new
QueryBuilder(`${window.location.protocol}/${window.location.host}${window.location.pathname}`);
      queryBuilder.addParam("schema", this.schema).addParam("search", this.search);
      if (this.page > 1) queryBuilder.addParam("page", this.page);

      history.pushState({ path: queryBuilder.getUrl() }, "", queryBuilder.getUrl());
    }
  }

  GenerateQueryURL() {
    const queryBuilder = new QueryBuilder('/api/glossary');
    queryBuilder.addParam('page', this.page).addParam('search', this.search).addParam('schema', this.schema);
    return queryBuilder.getUrl();
  }
}

const NOTIFICATION_TYPE_INFO = 1;
const NOTIFICATION_TYPE_WARNING = 2;

```

```

const NOTIFICATION_TYPE_ERROR = 3;

class Notification {
  constructor(text, type, duration) {
    this.text = text ?? "Текст оповещения";
    this.type = type ?? NOTIFICATION_TYPE_INFO;
    this.duration = duration ?? 2.5;
  }

  Create(container) {
    let notificationElement = document.createElement("div");
    notificationElement.classList.add("notification");
    notificationElement.innerText = this.text;
    container.append(notificationElement);

    switch (this.type) {
      case NOTIFICATION_TYPE_INFO:
        notificationElement.classList.add("info");
        break;
      case NOTIFICATION_TYPE_WARNING:
        notificationElement.classList.add("warning");
        break;
      case NOTIFICATION_TYPE_ERROR:
        notificationElement.classList.add("error");
        break;
    }

    // HACK: Force browser to reflow the element by accessing offsetWidth to play the show up animation
    notificationElement.offsetWidth;
    notificationElement.classList.add("show");

    const closeNotification = () => {
      notificationElement.classList.remove("show");
      setTimeout(() => notificationElement.remove(), 250);
    };

    let closeTimeout = setTimeout(closeNotification, this.duration * 1000);

    notificationElement.addEventListener("click", () => {
      clearTimeout(closeTimeout);
      closeNotification();
    })
  }
}

class NotificationManager {
  static containerElement;

  static {
    this.containerElement = document.querySelector("#NotificationContainer");
  }

  static CreateNotification(text, type, duration) {
    let notification = new Notification(text, type, duration);
    notification.Create(this.containerElement);
  }
}

class GlossaryList {
  constructor() {
    this.response = null;
    this.isLoading = true;
  }
}

```

```

    this.requestSettings = new RequestSettings();
}

CheckScroll() {
    let itemsElement = document.querySelector("#items");

    if (itemsElement.clientHeight < itemsElement.scrollHeight) {
        itemsElement.classList.add("scroll");
    } else {
        itemsElement.classList.remove("scroll");
    }
}

AddItem(item) {
    if (item) {
        const itemsElement = document.querySelector("#items");

        const itemElement = document.createElement("div");
        itemElement.classList.add("item");

        const tagsElement = document.createElement("div");
        tagsElement.classList.add("tags");
        itemElement.append(tagsElement);

        const infoWrapElement = document.createElement("div");
        infoWrapElement.classList.add("info-wrap");
        itemElement.append(infoWrapElement);

        const topElement = document.createElement("div");
        topElement.classList.add("top");
        infoWrapElement.append(topElement);

        const labelElement = document.createElement("span");
        labelElement.classList.add("label");
        labelElement.title = item.glsr_column_name;
        labelElement.innerText = item.glsr_column_name;
        topElement.append(labelElement);

        const AddTag = (text) => {
            const tagElement = document.createElement("div");
            tagElement.classList.add("tag");
            tagElement.innerText = text;
            tagsElement.append(tagElement);
        };

        // Generate type name
        const typeName = item.glsr_column_type;
        let typeNameTag = typeName;

        if (typeName === "numeric" && item.glsr_numeric_precision !== null) {
            typeNameTag += ` (${item.glsr_numeric_precision}, ${item.glsr_numeric_scale})`;
        } else if (item.glsr_char_max_length !== null) {
            typeNameTag += ` (${item.glsr_char_max_length})`;
        }

        AddTag(typeNameTag);

        if (item.glsr_primary_position !== null) {
            AddTag(первичный ключ (${item.glsr_primary_position}));
        } else if (item.glsr_foreign_table !== null) {
            AddTag(внешний ключ к таблице ${item.glsr_foreign_schema}.${item.glsr_foreign_table});
        }
    }
}

```

```

if (item.glsr_nullable == "NO") {
  AddTag("not null");
}

labelElement.addEventListener("click", () => {
  // Copy tag text
  navigator.clipboard.writeText(item.glsr_column_name);
  NotificationManager.CreateNotification("Название столбца скопировано!");
});

const hintElement = document.createElement("span");
hintElement.title = `${item.glsr_schema_name}.${item.glsr_table_name}`;
hintElement.classList.add("hint");
hintElement.innerText = hintElement.title;
topElement.append(hintElement);

const descriptionElement = document.createElement("p");
descriptionElement.classList.add("description");
descriptionElement.innerText = item.glsr_column_description;
infoWrapElement.append(descriptionElement);

const separatorElement = document.createElement("div");
separatorElement.classList.add("horizontal-separator");
separatorElement.hidden = true;
itemElement.append(separatorElement);

const suggestionsElement = document.createElement("div");
suggestionsElement.classList.add("suggestions");

// Use this to make sure tag doesn't already exist
let filter = new GaussFilter();

const colors = [
  '0, 255, 184',
  '255, 161, 44',
  '255, 239, 44',
  '78, 230, 255',
  '255, 130, 130',
  '255, 176, 130',
  '53, 255, 19',
  '195, 255, 19'
];

const AddSuggestion = (tag) => {
  // Check if text looks similar to a column name
  if (tag.length > 2 &&
    !filter.exists(tag) &&
    !tag.match(/^[a-zA-Z_#]|\(|\)|\||\./))
  ) {
    const suggestionElement = document.createElement("div");
    suggestionElement.classList.add("suggestion");
    suggestionElement.innerText = tag;

    let randomColor = colors[Math.abs(HashString(tag)) % colors.length];
    suggestionElement.style = `--color: ${randomColor};`;

    suggestionElement.addEventListener("click", () => {
      // Copy tag text
      navigator.clipboard.writeText(tag);
      NotificationManager.CreateNotification("Предложенный вариант скопирован!");
    });
  }
}

```

```

        suggestionsElement.append(suggestionElement);
        filter.add(tag);
    }
};

const parseTagsRegex = (regex) => {
    let iterator = item.glsr_column_description.matchAll(regex);
    let match = iterator.next();

    while (!match.done) {
        AddSuggestion(match.value[1]);
        match = iterator.next();
    }
};

// Try to parse mssql names from the description
parseTagsRegex(/^[(+)\]/g);
parseTagsRegex(/([A-Я|A-Z|a-z]+_[^\s\];\;]+)/g);

let descriptionWords = item.glsr_column_description.split(" ");
if (descriptionWords.length == 1) {
    AddSuggestion(descriptionWords[0].toLowerCase());
} else if (descriptionWords.length >= 2 && descriptionWords.length <= 4) {
    AddSuggestion(descriptionWords.join("_").toLowerCase());
}

if (suggestionsElement.children.length > 0) {
    separatorElement.hidden = false;
    itemElement.append(suggestionsElement);
}

itemsElement.append(itemElement);
}
}

Rebuild() {
    const contentWrap = document.querySelector("#content-wrap");
    contentWrap.classList.remove("empty");

    if (this.response.detail != 'Invalid page.') {
        // Clear items container
        const itemsElement = document.querySelector("#items");
        itemsElement.innerHTML = "";

        if (this.response.results.length > 0) {
            // Has some elements
            for (let i = 0; i < this.response.results.length; i++) {
                this.AddItem(this.response.results[i]);
            }

            itemsElement.scrollTo({ top: 0, behavior: 'smooth' })
        } else {
            // Has no elements
            contentWrap.classList.add("empty");
        }

        this.CheckScroll();
        this.RebuildPagination(this.requestSettings.page, this.response.page.num_pages);
    } else {
        // Invalid page
        contentWrap.classList.add("empty");
    }
}

```

```

    }
}

AddPage(index, isActive, isEnabled = true, text = null) {
    const pagesElement = document.querySelector("#pages");

    const pageElement = document.createElement("button");
    pageElement.classList.add("page");
    pageElement.innerText = text ?? index;
    pageElement.disabled = !isEnabled;

    if (isActive) {
        pageElement.classList.add("active");
    } else {
        pageElement.addEventListener("click", () => {
            this.RebuildPagination(index, this.response.page.num_pages);
            this.requestSettings.page = index;
            this.requestSettings.UpdateHistoryURL();
            UpdateItems(this.requestSettings);
        })
    }

    pagesElement.append(pageElement);
}

RebuildPagination(current, maxNum) {
    const pagesElement = document.querySelector("#pages");
    pagesElement.innerHTML = "";

    // Total number of page buttons is MAX_PAGE_RANGE * 2
    const MAX_PAGE_RANGE = 2;

    let numPagesAtLeft = Math.min(Math.max(current - 1, 0), MAX_PAGE_RANGE);
    let numPagesAtRight = Math.min(Math.max(maxNum - current, 0), MAX_PAGE_RANGE);

    if (numPagesAtLeft + numPagesAtRight > 0 && numPagesAtLeft + numPagesAtRight + 1 != maxNum) {
        if (numPagesAtRight < MAX_PAGE_RANGE) {
            numPagesAtLeft += MAX_PAGE_RANGE - numPagesAtRight;
        } else if (numPagesAtLeft < MAX_PAGE_RANGE) {
            numPagesAtRight += MAX_PAGE_RANGE - numPagesAtLeft;
        }
    }

    // Prev button
    this.AddPage(current - 1, false, current > 1, '<');

    // Add previous pages
    for (let i = numPagesAtLeft; i >= 1; i--) {
        this.AddPage(current - i, false);
    }

    // Add the current page button
    this.AddPage(current, true);

    // Add next pages
    for (let i = 1; i <= numPagesAtRight; i++) {
        this.AddPage(current + i, false);
    }

    // Next button
    this.AddPage(current + 1, false, current < maxNum, '>');
}

```

```

GetTotalCount() {
  return this.response ? this.response.count : 0;
}

GetNextPageURL() {
  return this.response ? this.response.next : null;
}

GetPrevPageURL() {
  return this.response ? this.response.prev : null;
}
}

document.addEventListener("load", () => {
  const glossaryList = new GlossaryList();

  const SetIsLoading = (isLoading) => {
    const contentWrap = document.querySelector("#content-wrap");

    if (isLoading) contentWrap.classList.add("loading");
    else contentWrap.classList.remove("loading");

    glossaryList.isLoading = true;
  };

  const UpdateItems = (settings) => {
    SetIsLoading(true);

    fetch(settings.GenerateQueryURL()).then((res) => {
      return res.json();
    }).then((data) => {
      SetIsLoading(false);
      glossaryList.response = data;
      glossaryList.Rebuild();
    });
  };

  // Initialise input and select elements
  // Makes them add/remove "focused" class to parents when events occur
  let inputElements = document.querySelectorAll("input[type='text'],select");
  for (let i = 0; i < inputElements.length; i++) {
    let element = inputElements[i];
    let parent = element.parentElement;

    if (parent) {
      element.addEventListener("focus", () => parent.classList.add("focused"));
      element.addEventListener("blur", () => parent.classList.remove("focused"));
    }
  }

  // Initialise clear search button
  let searchTimeout = null;
  const clearSearchButton = document.querySelector("#button-clear-search");

  clearSearchButton.addEventListener("click", (e) => {
    if (searchTimeout !== null) {
      clearTimeout(searchTimeout);
      searchTimeout = null;
    }

    document.querySelector("#search-input").value = "";
  });

```

```

glossaryList.requestSettings.search = "";
glossaryList.requestSettings.page = 1;
glossaryList.requestSettings.UpdateHistoryURL();
UpdateItems(glossaryList.requestSettings);
clearSearchButton.classList.add("hidden");
});

// Initialise history
const NO_FILTER_NAME = "NoSchemaFilter";
let isFirstPageLoad = true;

const OnPageLoad = () => {
    glossaryList.requestSettings.Reset();

    // Restore value from the query in url
    const searchElement = document.querySelector("#search-input");
    searchElement.value = glossaryList.requestSettings.search;

    if (glossaryList.requestSettings.search.trim().length > 0) {
        clearSearchButton.classList.remove("hidden");
    }

    if (!isFirstPageLoad) {
        // Don't need to set value before filters are loaded
        // The requested schema will be set as selected when they are loaded
        const filterElement = document.querySelector("#schema-filter");
        const querySchemaName = glossaryList.requestSettings.schema.trim();
        filterElement.value = querySchemaName.length != 0 ? querySchemaName : NO_FILTER_NAME;
    }

    UpdateItems(glossaryList.requestSettings);
    isFirstPageLoad = false;
};

// Initialise refresh button
document.querySelector("#buttonUpdate").addEventListener("click", () => {
    NotificationManager.CreateNotification("Обновление...");
    OnPageLoad();
});

// When url is changing with the back/next buttons, query items again
window.addEventListener("popstate", OnPageLoad);

// Initialise search
const searchElement = document.querySelector("#search-input");

if (searchElement) {
    searchElement.addEventListener("input", (e) => {
        let oldValue = glossaryList.requestSettings.search;
        let newValue = e.target.value.trim();

        if (newValue.length > 0) {
            clearSearchButton.classList.remove("hidden");
        } else {
            clearSearchButton.classList.add("hidden");
        }

        if (oldValue != newValue) {
            glossaryList.requestSettings.search = newValue;

            if (searchTimeout !== null) {
                clearTimeout(searchTimeout);
            }
        }
    });
}

```

```

        searchTimeout = null;
    }

    searchTimeout = setTimeout(() => {
        // Update url, do query
        glossaryList.requestSettings.page = 1;
        glossaryList.requestSettings.UpdateHistoryURL();
        UpdateItems(glossaryList.requestSettings);
        searchTimeout = null;
    }, 250);
    }
})
}

// Initialize schema filter
const filterElement = document.querySelector("#schema-filter");

if (filterElement) {
    const CreateOption = (value, name = undefined) => {
        let optionElement = document.createElement("option");
        optionElement.value = value;
        optionElement.innerText = name ?? value;

        if (glossaryList.requestSettings.schema === value) {
            // Set the option as selected if it was specified in the url
            optionElement.selected = true;
        }

        filterElement.append(optionElement);
    };

    const RemoveAllOptions = () => {
        filterElement.innerHTML = "";
    };

    CreateOption(NO_FILTER_NAME, "Загрузка...");

    fetch("/api/schemas").then(async (response) => {
        const possibleSchemas = await response.json();

        RemoveAllOptions();
        CreateOption(NO_FILTER_NAME, "Фильтрация по схеме");

        // Create 'option' elements
        for (let i = 0; i < possibleSchemas.length; i++) {
            CreateOption(possibleSchemas[i]);
        }

        filterElement.addEventListener("change", () => {
            const newValue = filterElement.value.trim();
            glossaryList.requestSettings.schema = (newValue !== NO_FILTER_NAME) ? newValue : "";

            // Apply filter and do query again
            glossaryList.requestSettings.page = 1;
            glossaryList.requestSettings.UpdateHistoryURL();
            UpdateItems(glossaryList.requestSettings);
        });
    });
}

// Check scroll when window resizing

```

```
window.addEventListener("resize", glossaryList.CheckScroll());  
  
    // Load items  
    OnPageLoad();  
});  
}());
```